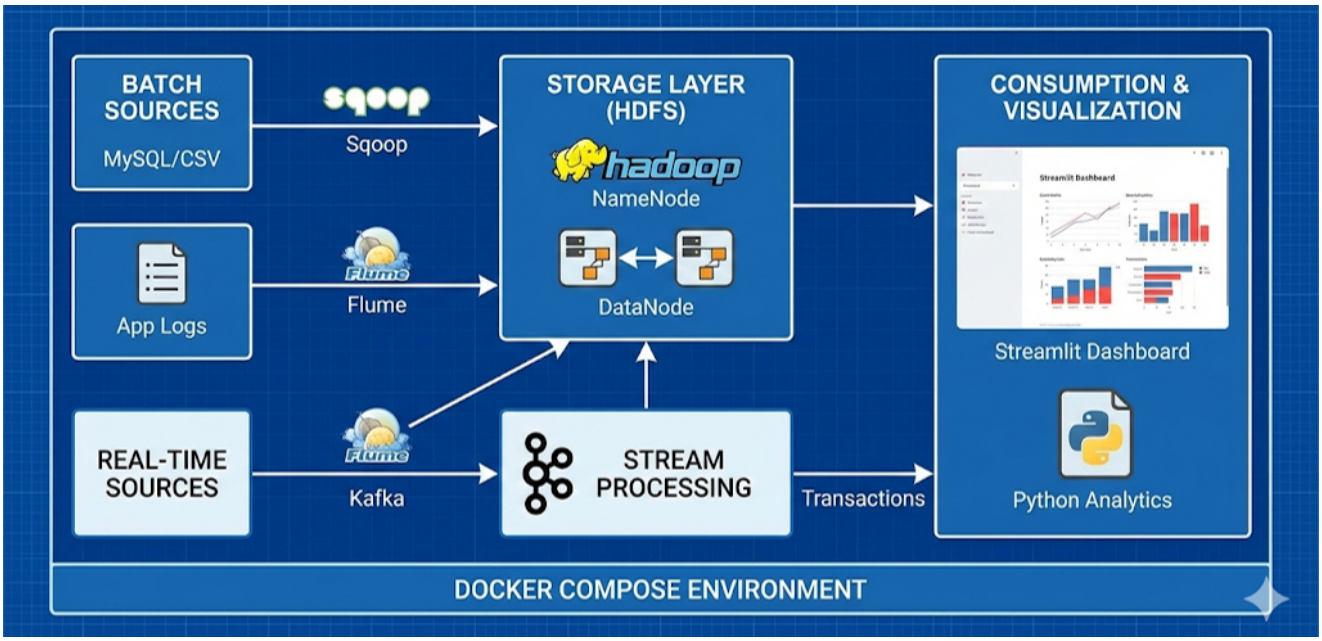
Workshop: Procesamiento de Big Data - Despliegue de Arquitecturas Modernas en Entornos Dockerizados

El objetivo de este proyecto es construir un pipeline de Big Data escalable capaz de manejar diferentes paradigmas de procesamiento de datos. Demuestra la integración de sistemas heterogéneos (SQL, NoSQL y flujos de eventos) en una plataforma de datos coherente mediante el uso de contenedores.

Arquitectura del Sistema

La solución se despliega de forma aislada utilizando Docker Compose y se divide en cuatro capas principales:

- **Ingesta Batch**: Sqoop realiza transferencias masivas de datos históricos desde una base de datos relacional (MySQL) hacia el Data Lake.
- **Ingesta en Tiempo Real**: Kafka maneja la transmisión de transacciones en vivo a través de un tópico dedicado llamado `transactions`.
- **Ingesta de Logs**: Flume captura continuamente los registros de la aplicación en tiempo real (haciendo "tail" a un archivo de log) y los archiva en HDFS.
- **Almacenamiento**: Hadoop HDFS (compuesto por un NameNode y un DataNode) funciona como el repositorio central distribuido.
- **Visualización y Consumo**: Un dashboard interactivo construido en Python con Streamlit unifica la visualización de datos históricos y en tiempo real.



Infraestructura y Orquestación

- **Docker compose:** Es la plataforma de contenerización que orquesta todo el stack de tecnologías. Permite empaquetar cada servicio con sus dependencias exactas, garantizando que el entorno sea reproducible y aislado, evitando conflictos de puertos en la máquina anfitriona.

PROF

Capa de Streaming y Mensajería (Speed Layer)

- **Kafka:** Es una plataforma de transmisión de eventos en tiempo real. Actúa como el sistema nervioso central para los datos en vivo, recibiendo las transacciones generadas al instante y distribuyéndolas a los consumidores a través de canales llamados "tópicos" (como el tópico `transactions`).
- **Zookeeper:** Es un servicio de coordinación distribuida esencial para que Kafka funcione. Se encarga de mantener la configuración del clúster, sincronizar los nodos, elegir a los líderes de partición y registrar la disponibilidad de los "brokers" de Kafka.
- **Kafka UI:** Es una interfaz web gráfica que facilita la gestión y el monitoreo del clúster de Kafka. Permite a los usuarios inspeccionar visualmente los tópicos, verificar los mensajes entrantes y administrar los brokers sin necesidad de usar la línea de comandos.

Capa de Almacenamiento Distribuido (Data Lake)

- **Hadoop HDFS (Hadoop Distributed File System):** Es el sistema de archivos distribuido que proporciona almacenamiento escalable para el lago de datos. Su arquitectura se divide en dos roles principales:
- **NameNode:** Es el nodo maestro o "cerebro" del sistema. No guarda los datos en sí, sino los metadatos: mantiene el árbol de directorios y sabe exactamente en qué DataNode está guardado cada fragmento de información.
- **DataNode:** Son los nodos trabajadores que almacenan físicamente los bloques de datos distribuidos.

Fuentes de Datos e Ingesta (Batch Layer)

- **MySQL:** Es un sistema de gestión de bases de datos relacionales tradicional. En este diseño, actúa como el origen estructurado que contiene los datos históricos de ventas acumulados.
- **Sqoop:** Es una herramienta diseñada para transferir eficientemente datos masivos (bulk data) entre bases de datos relacionales (MySQL) y el entorno de Hadoop (HDFS). Se encarga de la ingesta "Batch" del sistema.
- **Flume:** Es un servicio diseñado para recopilar, agregar y mover grandes cantidades de datos de registro (logs) hacia HDFS. En este pipeline, está configurado para "escuchar" continuamente un archivo de texto (`app.log`) y enviar cada nueva línea de registro al sistema de archivos distribuido.

PROF

Capa de Consumo y Servicio (Serving Layer)

- **Streamlit:** Es un framework de Python utilizado para construir rápidamente la aplicación del dashboard analítico. Permite crear interfaces interactivas para visualizar los insights unificados, conectándose tanto a la base de datos MySQL (para la historia) como a Kafka (para el flujo en tiempo real).

Requisitos Previos

- **Docker Desktop** instalado y en ejecución.
- **Python 3.9+** instalado en la máquina anfitriona para ejecutar los scripts locales de generación de datos.

Ejecución del Workshop (Paso a Paso)

1. Inicializar la Infraestructura

El primer paso es levantar todos los servicios (Zookeeper, Kafka, Hadoop, MySQL, Flume y el Dashboard). Abre una terminal en el directorio del proyecto y ejecuta:

```
conda create -n sem_GDM pandas sqlalchemy pymysql kafka-python
conda activate sem_GDM
docker-compose up -d --build
```

2. Cargar la Base de Datos (Carga Inicial)

Antes de procesar la información histórica, la base de datos MySQL necesita contener datos. Ejecuta el script de configuración para cargar el archivo CSV local hacia MySQL:

```
python datagen/setup_db.py
```

Esto conecta con MySQL en el puerto `13307` y crea la tabla `sales`.

3. Procesamiento Batch (ETL con Sqoop)

Una vez que MySQL tiene los datos, muévelos al sistema de archivos distribuido de Hadoop. Ejecuta el script de importación directamente dentro del contenedor de Sqoop:

```
docker exec sqoop sh /scripts/run_sqoop_import.sh
```

Esto lanza un trabajo que transfiere los registros a la ruta `/user/sqoop/sales` en HDFS.

4. Generación de Datos en Tiempo Real

Para simular el tráfico en vivo que alimentará a Kafka y Flume, necesitas iniciar el script productor y mantenerlo en ejecución en una terminal separada:

```
python datagen/producer.py
```

Este script cumple una doble función: envía objetos JSON al tópico `transactions` en Kafka y escribe registros de texto en `logs/app.log`. Flume detectará estas nuevas líneas en el log y las enviará a

`/user/flume/logs/` *en HDFS de forma automática.*

5. Monitoreo y Análisis

Con los datos fluyendo por el pipeline, abre el navegador web para consumir los resultados. Dirígete a `http://localhost:18501` para acceder al dashboard de Streamlit. Desde allí puedes verificar la salud del sistema (Zookeeper), observar el flujo en vivo de Kafka y recargar las analíticas históricas extraídas de MySQL.

Interfaces y Puertos Expuestos

Para auditar el correcto funcionamiento de cada componente, puedes acceder a las siguientes interfaces web desde tu navegador:

Servicio	URL Local	Propósito
Streamlit Dashboard	`http://localhost:18501`	Visualizar KPIs, gráficos históricos y flujo de transacciones en vivo.
Hadoop NameNode	`http://localhost:19870`	Navegar por el sistema de archivos HDFS y verificar las salidas de Sqoop y Flume.
Kafka UI	`http://localhost:18080`	Inspeccionar los brokers, administrar tópicos y visualizar los mensajes entrantes.